Next.js Project Structure — Master Overview

> Companion to the official page: https://nextjs.org/docs/app/getting-started/project-structure
>
> This is the "big picture" guide. Each concept also has its own focused `GUIDE.md` / `GUIDE.pdf` colocated next to the live demo inside `nextjs-structure-lab/src/app/<concept>/`.

---

1. The mental model

Next.js (App Router) is **file-system based routing**. You express your app's URLs and UI by *creating folders and special files* — not by registering routes in code.

Two rules explain almost everything:

1. **A folder = a URL segment.** `app/blog/authors/` → `/blog/authors`.
2. **A folder becomes a public page only when it contains a `page.js` / `page.tsx` (UI) or `route.js` / `route.ts` (API).** Folders without those are just organization — safe to put components, tests, and helpers inside them ("colocation").

Everything else (dynamic routes, route groups, private folders, parallel routes, intercepting routes, metadata files) is a *naming convention* layered on top of those two rules.

---

2. Top-level folders

Folder	Purpose
`app`	App Router — the modern routing system (what this lab uses).
`pages`	Pages Router — the older routing system (see `examples/pages-router`).
`public`	Static assets served from `/` (e.g. `public/logo.png` → `/logo.png`).
`src`	Optional. Holds app code (incl. `app/`) to separate it from config files at the repo root. **This lab uses `src/`.**

3. Top-level files (configuration)

File	What it does
`next.config.ts`	Next.js configuration (TypeScript config is supported).
`package.json`	Dependencies + scripts (`dev`, `build`, `start`).
`proxy.ts`	**Next 16+** request proxy — runs before requests. (Replaces the old `middleware.ts`.)
`instrumentation.ts`	OpenTelemetry / startup instrumentation hook.
`.env`, `.env.local`	Environment variables (NOT committed — only `.env.example` is).
`eslint.config.mjs`	ESLint configuration.
`tsconfig.json`	TypeScript configuration (incl. the `@/*` import alias).
`next-env.d.ts`	Auto-generated TS types for Next.js (do not edit / commit).
`.gitignore`	Excludes `node_modules`, `.next`, real `.env` files, etc.

4. Routing files (the "special" files)

These filenames have built-in meaning inside any route folder:

File	Role
`layout`	Shared UI that wraps a segment **and all its children**. Persists across navigation (does NOT re-render).
`template`	Like `layout`, but **re-mounts** on every navigation (fresh state).
`page`	The unique UI of a route — makes it publicly accessible.
`loading`	Instant loading UI (a React **Suspense** boundary).
`error`	Error UI for a segment (a React **error boundary**). Must be a Client Component.
`global-error`	Error boundary for the **root layout** itself.
`not-found`	UI shown when `notFound()` is called or a route is unmatched.
`route`	An API endpoint (`GET`, `POST`, ...). A folder can have `page` **or** `route`, not both.
`default`	Fallback UI for an unmatched **parallel route** slot.

➡ Live demo: `src/app/routing-files/` and `src/app/component-hierarchy/`.

5. Nested & dynamic routes

- **Nested:** nest folders to nest URL segments; layouts wrap their children.
- **Dynamic:** brackets parameterize a segment. Read values from the `params` prop (which is a **Promise in Next 15/16 — you must `await` it**).

Folder	Matches
`blog/[slug]`	`/blog/my-post` (one segment)
`shop/[...slug]`	`/shop/a`, `/shop/a/b/c` (catch-all)
`docs/[[...slug]]`	`/docs`, `/docs/a/b` (optional catch-all)

➡ Live demos: `src/app/blog/`, `src/app/shop/`, `src/app/docs/`.

6. Organization without touching the URL

Convention	Syntax	Effect
Route group	`(group)`	Folder name omitted from the URL. Used to share a layout among some routes, or to split into sections.
Private folder	`_folder`	Opted out of routing entirely. For components/helpers.
Colocation	(any file)	Non-`page`/`route` files in a route folder are never routable, so they're safe to colocate.

➡ Live demos: `src/app/(marketing)/`, `src/app/(store)/`, `src/app/private-folders/`, `src/app/colocation/`.

7. Advanced UI routing

- **Parallel routes** `@slot`: render multiple pages in the same layout simultaneously (e.g. dashboard with team + analytics panels). Needs `default.tsx` for unmatched slots. ➡ `src/app/dashboard/`.

- **Intercepting routes** `(.)`, `(..)`, `(...)`: render another route *inside the current layout* (classic "open a detail as a modal" pattern). ➡ `src/app/gallery/`.

8. Metadata files

Drop these special files in `app/` and Next.js wires up the `<head>` / serves the file automatically:

- Icons: `favicon.ico`, `icon.(tsxlpng)`, `apple-icon.(tsxlpng)`
- Social: `opengraph-image.tsx`, `twitter-image.tsx`
- SEO: `sitemap.ts` → `/sitemap.xml`, `robots.ts` → `/robots.txt`

➡ Files live at `src/app/`; indexed by `src/app/metadata/`.

9. Component hierarchy (render order)

For a given segment, Next renders these special files nested in this order:

```
layout
└── template
    └── error (boundary)
        └── loading (suspense)
            └── not-found (boundary)
                └── page  (or a nested layout, recursively)
```

➡ Live demo: `src/app/component-hierarchy/`.

10. Organization strategies (pick one, be consistent)

1. **Outside `app`** — keep `app/` purely for routing; put `components/`, `lib/`, etc. in the project root (or `src/`).
2. **Inside `app`** — put shared `components/`, `lib/` at the top of `app/`.
3. **Split by feature/route** — keep shared code at the root of `app/`, and colocate route-specific code in the route folder.

> **This lab uses strategy #3** inside `src/app`: shared bits at the top, and each concept's demo code (and its `GUIDE`) colocated in the concept's folder.

11. Multiple root layouts

To give different sections completely different shells, **delete the top-level `layout`** and add a `layout` (with its own `<html>`/`<body>`) inside each route group. Because that conflicts with this lab's single shared shell, it lives in its own tiny app: `examples/multiple-root-layouts/`.

How this repo is organized

```
nextjs-structure-lab/   ← one runnable app demonstrating ~all concepts
examples/               ← tiny standalone apps for concepts that need isolation
guides/                 ← this overview + shared PDF stylesheet
scripts/build-pdfs.mjs  ← regenerates every GUIDE.pdf from its GUIDE.md
```

See the root `README.md` for run instructions.